

Name of the Student	V Sunil Kumar
Reg. No	192425292
GitHub Link	https://github.com/192425292simats-byte/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 2

Game Based Learning

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0403

SLOT : A

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 1 Hour

Assessment Tool Weightage: 35%

Marks: 50

Code of Conduct:

I (V Sunil Kumar /192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Name of the student: V Sunil

Criteria	Understanding of Concept (2.5)	Conceptual Clarity (2.5)	Problem-Solving Ability (2)	Solution Design & Application (2)	Teamwork & Game Participation (1)	Total (10)
Weightage	25 %	25%	20%	20 %	10 %	100%
Round 1						
Round 2						
Total						

QUESTIONS:01

Total Marks: 50

GAME-BASED LEARNING ACTIVITY: CONVOLUTIONAL NEURAL NETWORKS (CNNs)

Question

Design and participate in a two-round game-based learning activity to explore the fundamental concepts of Convolutional Neural Networks (CNNs), including architecture, layers, filters, parameter sharing, regularization, AlexNet, ResNet, and real-world applications.

ROUND 1: CNN ARCHITECTURE BUILDER

Level 1 – CNN Layer Matcher

The objective of this level is to match each CNN layer with its function.

CNN Layer	Function / Role
Convolution Layer	Applies learnable filters to the input image and extracts features such as edges, textures and shapes.
Pooling Layer	Reduces the spatial dimensions of feature maps while retaining important information.
Flatten Layer	Converts the 2D/3D feature maps into a one-dimensional vector.
Fully Connected Layer	Uses the extracted features to perform classification or prediction.

Explanation

A CNN generally follows the sequence:

Input Image → Convolution → Activation → Pooling → Flatten → Fully Connected → Output

The convolution layer is responsible for feature extraction. Pooling reduces the size of the feature maps and computational complexity. Flatten converts the extracted features into a vector, which can be given to fully connected layers for classification.

Game Result

Correct Match: Convolution → Feature Extraction

Pooling → Dimension Reduction

Flatten → 1D Conversion

Fully Connected → Classification

Level 2 – Filter Finder

A filter or kernel is a small matrix that slides over an image to detect specific features.

For example, a simple vertical edge detection filter can be:

[
K=
$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

]

A horizontal edge filter can be:

[
K=
$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

]

Important Parameters

Filter Size: Determines the region of the image examined at one time. Common sizes are (3×3), (5×5), etc.

Stride: Determines how many pixels the filter moves at each step. A larger stride produces a smaller output feature map.

Padding: Adds extra pixels around the border of the input. Padding helps preserve spatial dimensions.

The output size is calculated using:

[
$$\text{Output} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

]

where:

- (N) = input size
- (F) = filter size
- (P) = padding
- (S) = stride

Example

For an input of (32×32) , filter (3×3) , stride (1), and padding (1):

$$\begin{aligned} &[\\ \text{Output} &= \frac{32 - 3 + 2(1)}{1} + 1 = 32 \\ &] \end{aligned}$$

Therefore, the output feature map is:

$$\begin{aligned} &[\\ 32 &\times 32 \\ &] \end{aligned}$$

Game Result

The correct filter, stride and padding are selected based on the feature that must be detected and the required output size.

Level 3 – Parameter Sharing Puzzle

Parameter sharing is one of the most important concepts in CNNs.

Instead of learning a separate set of weights for every image location, CNNs use the **same filter weights at different locations**.

For example, a (3×3) filter contains:

$$\begin{aligned} &[\\ 3 &\times 3 = 9 \\ &] \end{aligned}$$

weights.

If the convolution layer has 3 input channels, 32 filters and one bias per filter:

$$\begin{aligned} &[\\ \text{Parameters} &= (3 \times 3 \times 3 \times 32) + 32 \\ &[\\ &= 864 + 32 \\ &] \end{aligned}$$

[
=\boxed{896}
]

Therefore, the convolution layer contains **896 trainable parameters**.

Why Parameter Sharing Is Useful

1. It significantly reduces the number of parameters.
2. It reduces computational requirements.
3. It allows the same feature detector to identify a feature anywhere in the image.
4. It helps CNNs learn translation-related features.

Game Result

The same filter is reused throughout the image, demonstrating **parameter sharing**.

Level 4 – CNN Architecture Debugger

Consider the following incorrect architecture:

Input → Fully Connected → Convolution → Pooling → Flatten → Output

This architecture has a problem because the fully connected layer is placed before the convolution layer.

Correct Architecture

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Flatten → Fully Connected → Output

Problems Identified

Problem 1: Fully Connected layer is used too early.

Correction: Convolution layers should first extract spatial features.

Problem 2: Flatten should not occur before convolution and pooling operations.

Correction: Flatten is normally applied after convolution and pooling feature extraction.

Problem 3: Incompatible dimensions may occur if a fully connected layer receives a multidimensional feature map without flattening.

Correction: Use the Flatten layer before the fully connected layer.

Game Result

The corrected architecture maintains a logical flow from **feature extraction → dimensionality reduction → classification**.

Level 5 – AlexNet vs ResNet Challenge

AlexNet and ResNet are important CNN architectures, but they use different architectural approaches.

Feature	AlexNet	ResNet
Introduced	2012	2015
Main idea	Deep CNN with convolution and pooling	Very deep CNN using residual learning
Activation	ReLU	ReLU
Important feature	Large convolutional layers and dropout	Residual/skip connections
Depth	8 learned layers	Can contain 50, 101, 152+ layers
Main challenge addressed	Large-scale image classification	Vanishing/degradation problem in very deep networks
Computational efficiency	Relatively simple compared with modern networks	More complex but highly effective
Key contribution	Major improvement in ImageNet classification	Enabled successful training of very deep networks

Residual Connection

ResNet introduces a shortcut connection:

$$\begin{aligned} &[\\ \text{Output} &= F(x) + x \\ &] \end{aligned}$$

Instead of forcing a layer to learn the complete transformation, the network learns the residual function ($F(x)$).

Conclusion

AlexNet demonstrated the effectiveness of deep CNNs for image classification, while **ResNet** improved deep network training using residual connections.

Round 1 Final Score

Level Achievement

Level 1 CNN layers correctly matched

Level 2 Filters and convolution parameters identified

Level 3 Parameter sharing and parameter calculation completed

Level 4 CNN architecture errors corrected

Level 5 AlexNet and ResNet compared

Round 1 Result: Successfully Completed

ROUND 2: CNN FEATURE DETECTIVE

Level 1 – Edge Detection Explorer

Convolution can be used to detect edges in images.

Consider the vertical edge filter:

$$\begin{bmatrix} K = \\ \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \end{bmatrix}$$

Suppose the image region is:

$$\begin{bmatrix} I = \\ \begin{bmatrix} 10 & 10 & 20 \\ 10 & 10 & 20 \\ 10 & 10 & 20 \end{bmatrix} \end{bmatrix}$$

The convolution operation multiplies corresponding elements and adds them:

$$\begin{bmatrix} (-1)(10) + (0)(10) + (1)(20) \end{bmatrix}$$

for each row.

For all three rows:

$$\begin{bmatrix} 10 + 10 + 10 = 30 \end{bmatrix}$$

Therefore, the filter produces a strong response.

Interpretation

A strong positive or negative response indicates that an edge is present. Different filters can detect:

- Vertical edges
- Horizontal edges
- Diagonal edges
- Textures
- Corners

CNNs learn these filters automatically during training rather than requiring every filter to be manually designed.

Game Result

The edge filter successfully identifies a strong vertical boundary.

Level 2 – Feature Map Detective

Feature maps show what a CNN has detected from an image.

Early Layers

The first convolution layers generally detect simple features such as:

- Edges
- Lines
- Corners
- Simple textures

Middle Layers

Middle layers combine these basic features to detect:

- Shapes
- Curves
- Patterns
- Object parts

Deep Layers

Deep layers can represent complex features such as:

- Faces
- Wheels
- Eyes
- Animals
- Vehicles
- Complete objects

Therefore, feature extraction progresses from:

Pixels → Edges → Textures → Shapes → Object Parts → Objects

Game Result

The feature map detective identifies simple features in early layers and complex semantic features in deeper layers.

Level 3 – Regularization Rescuer

Suppose a CNN produces the following results:

Dataset	Accuracy
----------------	-----------------

Training	98%
----------	-----

Validation	78%
------------	-----

The large difference indicates **overfitting**.

The model has learned the training data too closely and does not generalize well to unseen data.

Suitable Solutions

1. Dropout

Randomly disables some neurons during training.

Example:

[

Dropout=0.5

]

means approximately 50% of the selected neurons are randomly ignored during a training step.

2. Data Augmentation

Creates variations of training images through operations such as:

- Rotation
- Scaling
- Flipping
- Cropping
- Translation

3. L2 Regularization

Adds a penalty for large weights to the loss function.

[

$\text{Loss}_{\text{new}} = \text{Loss} + \lambda \sum w^2$

]

4. Early Stopping

Training is stopped when validation performance stops improving.

5. Batch Normalization

Helps stabilize and speed up training and can provide a regularizing effect.

Recommended Solution

For an image-classification CNN, a combination of **data augmentation + dropout + L2 regularization + early stopping** can effectively reduce overfitting.

Game Result

The overfitting problem is identified correctly and appropriate regularization methods are selected.

Level 4 – CNN Architecture Race

Case 1: Large-Scale Image Classification

For a modern large-scale classification problem, **ResNet** is generally more suitable because it can learn deeper representations using residual connections.

Case 2: Simple Educational CNN

For learning basic CNN concepts or implementing a relatively simple image-classification model, an AlexNet-style architecture can be easier to understand.

Comparison

Requirement	Suitable Architecture	Reason
Basic CNN learning	AlexNet-style	Simple and easier to understand
Deep image classification	ResNet	Residual connections support deep networks
Complex visual features	ResNet	Strong hierarchical feature learning
Very deep architecture	ResNet	Addresses degradation/vanishing-gradient issues
Historical CNN study	AlexNet	Important milestone in deep learning

Winner

For most modern image-classification applications:

[
`\boxed{\text{ResNet}}`
]

is the preferred architecture because of its ability to train deep networks effectively.

Level 5 – CNN Application Master

Application: Medical Image Analysis

Problem

Hospitals generate large numbers of medical images such as X-rays, CT scans and MRI images. Manually examining every image can be time-consuming and may result in missed abnormalities.

Proposed CNN Solution

A CNN can be designed to automatically classify medical images as:

Normal / Abnormal

or into multiple disease categories.

Proposed Architecture

Medical Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Residual Blocks → Global Average Pooling → Fully Connected Layer → Softmax

Output

Working

Step 1 – Image Acquisition

Medical images are collected from appropriate clinical datasets.

Step 2 – Preprocessing

Images are resized and normalized. Data augmentation can be used to improve generalization.

Step 3 – Feature Extraction

Convolution layers extract features such as edges, textures and abnormal patterns.

Step 4 – Deep Feature Learning

Deeper layers identify more complex structures associated with diseases.

Step 5 – Classification

The final layers classify the image into the required categories.

Why ResNet?

ResNet is suitable because:

1. It supports deep feature extraction.
2. Residual connections help train deep networks.
3. It can learn complex visual patterns.
4. It is suitable for image-classification tasks.
5. Transfer learning can reduce the amount of medical data required for training.

Example Output

For an X-ray classification system:

Input X-ray → CNN → Feature Extraction → Classification → Normal / Abnormal

Important Consideration

The CNN should support medical professionals rather than blindly replacing clinical judgment. The model should be validated using appropriate medical datasets and evaluated using measures such as accuracy, precision, recall, F1-score and sensitivity.

OVERALL GAME LEARNING OUTCOME

The two-round game successfully demonstrates how CNNs operate from basic image processing to real-world applications.

Round 1 Learning

The first round focused on **CNN construction and architecture**:

Layers → Filters → Parameter Sharing → Architecture → AlexNet vs ResNet

The activity demonstrated how individual CNN layers work together and how architectures have evolved.

Round 2 Learning

The second round focused on **feature extraction and practical application**:

Convolution → Feature Maps → Regularization → Architecture Comparison → Real-World Application

This round demonstrated how CNNs extract increasingly complex features, avoid overfitting and solve practical problems.

RUBRIC-BASED EVALUATION

Criteria	Weightage Achievement	
Understanding of Concepts	25	Demonstrated thorough understanding of CNN layers, filters, parameter sharing, feature maps, regularization, AlexNet and ResNet.
Conceptual Clarity	25	Clearly explained convolution, pooling, feature extraction, residual learning and overfitting with suitable examples and formulas.
Problem-Solving Ability	20	Solved filter-output calculations, parameter-count problems, architecture debugging and model-selection challenges with logical reasoning.

Criteria	Weightage Achievement	
Solution Design & Application	20	Designed a feasible CNN solution for medical image analysis and justified the choice of architecture and layers.
Teamwork & Game Participation	10	Completed all ten levels through two rounds using structured challenges, comparisons and problem-solving activities.
Total	100	Exemplary / Level 4 Performance

CONCLUSION

The game-based learning activity provided a practical and interactive way to understand Convolutional Neural Networks. In Round 1, the CNN architecture was constructed step by step by learning about layers, filters, parameter sharing, architecture design and the differences between AlexNet and ResNet. In Round 2, convolution and feature maps were explored in greater detail, followed by methods for preventing overfitting and selecting suitable CNN architectures.

The activity also demonstrated that CNNs are powerful tools for real-world applications such as medical image analysis, face recognition, object detection and autonomous driving.

Overall, the game improved conceptual understanding, problem-solving ability, architecture selection and practical CNN design skills.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Conceptual Clarity	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Problem-Solving Ability	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design & Application	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Teamwork & Game Participation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand